Série nř1 | TD/TP – Durée : 2 h

Synchrone, asynchrone et fonctions comme valeurs

Université Hassan I – Faculté des Sciences et Techniques de Settat
Département Physique Appliquée – Laboratoire LAVETE
Pr. Abdelfattah Abassi Année universitaire 2026–2027

Chapitre Synchrone et asynchrone ; Fonctions comme valeurs

Objectifs Distinguer exécution synchrone/bloquante et asynchrone/non bloquante ; mesurer un temps d'exécution ; manipuler les fonctions anonymes, flèche, d'ordre supérieur et les paramètres nommés.

Matériel SDK Dart installé, éditeur (VS Code), terminal ; les programmes se lancent avec `dart run fichier.dart`.

Durée	Partie	Contenu
45 min	TD (papier)	vocabulaire, types de fonctions, prédiction de sorties, paramètres nommés
75 min	TP (machine)	chronométrage, fonctions d'ordre supérieur, mini-calculatrice

Partie A – TD (45 min)

TD 1 | Vocabulaire 10 min

a) Classez chaque situation en *synchrone* ou *asynchrone* : (1) attendre au guichet jusqu'à son tour ; (2) lancer une machine à laver et ranger la cuisine ; (3) appeler un ami et attendre qu'il décroche sans rien faire d'autre ; (4) commander un plat et s'asseoir pendant la préparation ; (5) calculer `3 * 4` dans un programme.

b) Classez en *rapide* ou *lent* : trier 10 nombres, télécharger une image, lire un capteur Bluetooth, comparer deux entiers.

c) Un écran affiche 60 images par seconde. Combien d'images environ sont perdues pendant une attente bloquante de 200 ms ?

d) Donnez la différence entre *concurrence* et *parallélisme*.

TD 2 | Pourquoi bloquer est un problème 5 min

Expliquez en quelques lignes pourquoi un `sleep(Duration(seconds: 2))` placé dans le code d'une application mobile **fige l'écran**. Quel adjectif décrit une opération qui ne fige pas le programme ?

TD 3 | Types de fonctions 8 min

Donnez le type fonction de chacune des expressions suivantes :

a) `(int a, int b) => a + b`

b) `(String s) => s.length`

c) `() => print('ok')`

d) `(double x) => x > 0`

e) `(List<int> l) => l.first`

TD 4 | Prédire la sortie — 10 min

```dart
int doubler(int x) => x * 2;
int appliquer(int v, int Function(int) f) => f(v);

void main() {
  var f = doubler;
  print(f(4));
  print(appliquer(5, (x) => x + 1));
  print(appliquer(3, doubler));
  var g = (int x) => x * x;
  print(appliquer(appliquer(2, g), g));
}
```

Que affiche ce programme ? Quelle différence y a-t-il entre `f` et `f(4)` ?

TD 5 | Paramètres nommés — 12 min

Soit la fonction :

```dart
void inscrire(String nom, {int age = 18, required String filiere}) {
  print('$nom, $age ans, filière $filiere');
}
```

a) Parmi les appels suivants, lesquels sont valides ? Justifiez les refus.
(1) `inscrire('Sara', filiere: 'MIP');` (2) `inscrire('Sara');` (3) `inscrire('Sara', 20, filiere: 'MIP');` (4) `inscrire('Lina', filiere: 'GI', age: 20);` (5) `inscrire(nom: 'Omar', filiere: 'GI');`

b) Que vaut `age` dans l'appel (1) ?

c) Réécrivez en syntaxe flèche : `bool estPair(int n) { return n % 2 == 0; }`. Pourquoi ne peut-on pas utiliser la flèche pour une fonction qui contient deux instructions ?

Partie B – TP (75 min)

Consignes

Créez un projet : `dart create serie1` puis travaillez dans le dossier `bin/`. Un exercice = un fichier (`tp1.dart`, `tp2.dart`, ...). Comparez toujours votre sortie à la sortie attendue.

TP 1 | Mesurer le blocage — 20 min

1. Écrivez un programme qui affiche « Message 1 », « Message 2 », « Message 3 » avec une pause de 1 s (`sleep` de `dart:io`) après chacun, et affiche la durée totale mesurée avec `Stopwatch`.
2. Exécutez ensuite, **à titre d'observation**, ce second programme et comparez l'ordre d'affichage avec celui du premier :

```dart
void main() {
  print('A');
  Future.delayed(Duration(seconds: 1), () => print('C'));
  print('B');
}
```

Sortie attendue :

```
Message 1
Message 2
Message 3
Durée totale : 3003 ms     (environ 3000 ms)
```

```
A
```

```
B
C                          (affiché après environ 1 s)
```

3. Expliquez par écrit pourquoi `C` apparaît *après* `B` dans le second programme, alors que `print('C')` est écrit avant `print('B')`.

TP 2 | Boîte à outils de fonctions — 25 min

Dans un même fichier :

1. Rangez dans une variable `triple` une fonction anonyme en syntaxe flèche qui renvoie le triple d'un entier ; affichez `triple(7)`.
2. Écrivez `void repeter(int n, void Function(int) action)` qui appelle `action(i)` pour `i` de 1 à `n`. Testez avec `(i) => print('Tour $i')` puis avec une fonction qui affiche le carré de `i`.
3. Écrivez `List<int> transformer(List<int> l, int Function(int) f)` qui **renvoie une nouvelle liste** des `f(x)` (avec une boucle `for` et `add`). Testez avec le carré, le double et `(x) => x + 100` sur `[1, 2, 3]`.
4. *Pour aller plus loin* : écrivez `bool Function(int) seuil(int s)` qui **renvoie une fonction** testant si un entier dépasse `s`. Testez : `var estGrand = seuil(10); print(estGrand(12));`

Sortie attendue :

```
21
Tour 1
Tour 2
Tour 3
1
4
9
[1, 4, 9]
[2, 4, 6]
[101, 102, 103]
true
```

TP 3 | Mini-calculatrice d'ordre supérieur — 30 min

1. Écrivez `void tester(String nom, double Function(double, double) op)` qui affiche `nom : résultat de op(12, 4)`.
2. Appelez `tester` quatre fois avec des fonctions anonymes : addition, soustraction, multiplication, division.
3. Écrivez `double Function(double, double) choisir(String symbole)` qui renvoie la fonction correspondant à `'+'`, `'-'`, `'*'` ou `'/'` (utilisez `switch`) ; pour un symbole inconnu, la fonction renvoyée donne `double.nan`.
4. Écrivez une fonction `afficherFiche` de signature ci-dessous et testez-la avec trois appels dont l'ordre des paramètres nommés varie :

```
void afficherFiche(String nom,
    {int age = 18, required String filiere, String ville = 'Settat'})
```

Sortie attendue :

```
addition : 16.0
soustraction : 8.0
multiplication : 48.0
division : 3.0
choisir('*')(6, 7) = 42.0
choisir('?')(6, 7) = NaN
```

À savoir

Rendu : les fichiers `.dart` des trois exercices du TP et les réponses écrites aux questions de TD non traitées en séance.